KitchenOS

Point of Sale & Kitchen Management System

Project Report — Software Design & System Engineering (SDSE), Semester 4

Team Members

• Param Khodiyar
• Adit Singh
• Rishita Boisnobi
• Pratiti Paul
• Kushagra Bhardwaj

1. Project Overview

KitchenOS is a comprehensive, full-stack Point of Sale (POS) and Kitchen Management System designed for cafes and restaurants. Built as an academic project for the Software Design & System Engineering course, it integrates real-world software architecture patterns with a production-grade technology stack. The system consolidates order taking, kitchen display management, inventory tracking, analytics, and multi-account financial recording into a single cohesive platform.

The platform is architected around three operational roles — Owner, Cashier, and Kitchen — each with scoped access and a tailored user interface.

1.1 Objectives

- Digitise café/restaurant operations eliminating paper-based order workflows.
- Provide real-time kitchen visibility through a dedicated Kitchen Display System (KDS).
- Enforce role-based access control (RBAC) to separate concerns between Owner, Cashier, and Kitchen staff.
- Surface analytics and financial reporting for data-driven business decisions.
- Demonstrate software design patterns (Singleton, Strategy) and SOLID principles in a production context.

1.2 Deployment Links

- **Frontend (Vercel):** https://kitchen-os-sdse.vercel.app
- **Backend (Render):** https://kitchenossdse.onrender.com
- **Source Repository:** https://github.com/paramkhodiyar/KitchenOS-SDSE
- **System Architecture (Excalidraw):** https://excalidraw.com/#room=b18d71771add5de4b4f0,1l4lNnrEFPMoBuh1d8_iOQ

2. Technology Stack

2.1 Frontend

Framework	Next.js 16 / React 19 (TypeScript)
Styling	Tailwind CSS v4 with CSS variables for theming
State Management	Zustand v5 — global stores for Cart and Auth state
UI Components	Radix UI primitives, Lucide React icons, custom component layer
Data Visualisation	Recharts v3 — revenue and order analytics charts
Animations	Framer Motion v12 for UI transitions
HTTP Client	Axios with a centralised API service layer
Notifications	React Hot Toast, Sonner for contextual alerts
Linting / Types	ESLint (Next.js config), TypeScript v5

2.2 Backend

Runtime	Node.js v18+ (ES Modules)
Framework	Express.js v5

Database	PostgreSQL (hosted on Render)
ORM	Prisma v6 — schema modelling, migrations, type-safe queries
Authentication	JWT (jsonwebtoken v9) + Bcrypt v6 for PIN hashing
Logging	Custom middleware for request and error logging (Winston-style)
Dev Tooling	Nodemon, tsx for live reloading

3. Architecture & Project Flow

KitchenOS is structured as a monorepo with two independent applications: a Next.js frontend SPA and an Express.js REST API backend. Communication between layers is exclusively via JSON over HTTP; no server-side rendering is used for application data.

3.1 High-Level Architecture

- Client (SPA) — Next.js app served on Vercel. All pages are client-rendered; Next.js is used for routing (App Router) and build tooling only.
- API Server — Express.js on Render exposes RESTful endpoints under /v1/. All protected routes require a valid JWT Bearer token.
- Database — PostgreSQL managed through Prisma ORM. Schema migrations are version-controlled in the repository.

3.2 Application Flow

The following end-to-end flow describes a typical café session:

Store Initialisation: The owner visits /setup, registers the store name, and sets role-specific PINs. A unique Store Code is generated and used for all subsequent logins.

Authentication: Users navigate to /login, enter the Store Code, select their role (Owner / Cashier / Kitchen), and authenticate via a PIN pad. On success, a JWT is issued and persisted to the Zustand auth store (localStorage-backed).

Order Lifecycle (Cashier): The Cashier opens the POS interface, browses or searches the product catalogue, adds items to the cart, applies quantity adjustments, and proceeds to checkout. A payment method (Cash, UPI) is selected. The system checks raw material availability and flags out-of-stock conflicts. Owners can override. On confirmation, an Order and Transaction are persisted.

Kitchen Display (Kitchen Role): The Kitchen interface polls for new orders and displays them in real time. Staff transition order status: Pending → Preparing → Ready → Completed. Visual urgency cues assist prioritisation.

Inventory & Reporting (Owner): The Owner monitors raw material stock levels, receives automated low-stock alerts, manages the product catalogue, reviews revenue trends (Daily / Weekly / Monthly), and accesses order completion and cancellation statistics.

3.3 Module Structure

Backend is organised as a feature-module architecture. Each domain (auth, products, orders, inventory, accounts, reports) owns its own routes, controller, service, and schema files, following the Single Responsibility Principle:

- auth — Store setup, PIN unlock, JWT issuance, PIN reset
- products — CRUD for menu items, soft delete/archive
- orders — Order creation, item addition, status transitions
- inventory — Raw material management, availability computation

- accounts — Financial accounts (Cash, UPI, Bank) and transaction recording
- reports — Revenue, order statistics, and stock alert aggregations

4. System Design Concepts

4.1 Design Patterns

Singleton Pattern — Database Connection

Location: backend/src/config/prisma.js. The PrismaClient instance is created once at application startup and exported as a module-level singleton. This prevents connection pool exhaustion that would occur if a new client were instantiated per request. It is the central artery of all data access across every service module.

Strategy Pattern — Payment Processing

Location: backend/src/utils/paymentStrategy.js. KitchenOS supports multiple payment modes (CASH, UPI, BANK). Rather than embedding a conditional chain inside the transaction logic, the Strategy Pattern defines a common PaymentStrategy interface. CashPaymentStrategy and UPIPaymentStrategy implement the processPayment method independently, allowing the payment processor to swap algorithms at runtime without modifying the core order recording flow. New payment modes (e.g., Crypto) can be added by introducing a new strategy class — adherent to the Open/Closed Principle.

4.2 SOLID Principles

- Single Responsibility Principle (SRP): Every service file has a single bounded domain. orders.service.js manages order lifecycle; paymentStrategy.js manages payment processing. No cross-cutting logic is mixed.
- Open/Closed Principle (OCP): The payment strategy architecture is open for extension (add a new strategy class) but closed for modification (core transaction logic is untouched).
- Dependency direction: Controllers depend on services; services depend on the Prisma singleton. No circular dependencies.

4.3 OOP Concepts

- Encapsulation: Database mutation is hidden behind service functions such as recordOrderIncome, protecting schema integrity from direct manipulation.
- Polymorphism: The processPayment method in the Strategy pattern behaves differently per implementation (Cash vs UPI) while sharing a uniform call signature.
- Abstraction: Service layers abstract away Prisma query complexity from route controllers.

4.4 SDLC Approach

The project followed an Agile/Iterative SDLC:

- Requirement Analysis — Role identification (Owner, Cashier, Kitchen), feature scoping (POS, KDS, Inventory, Reports).
- Design — ER, Class, Sequence, and Use Case diagrams produced prior to implementation.
- Implementation — API-first backend development paired with a responsive Next.js frontend.
- Testing & Deployment — Iterative delivery; backend deployed on Render, frontend on Vercel.

4.5 Role-Based Access Control (RBAC)

Authentication is PIN-based per role. JWT tokens carry the role claim and are validated by the requireRole middleware on protected endpoints. The permission matrix is as follows:

Capability	Owner	Cashier	Kitchen
POS / Order Entry	Yes	Yes	No
Kitchen Display (KDS)	Yes	No	Yes
Order Status Update	Yes	No	Yes
Product / Menu Management	Yes	No	No
Inventory Management	Yes	Limited	No
Analytics & Reports	Yes	No	No
Account & Transactions	Yes	Yes	No
PIN Reset	Yes	No	No
Out-of-Stock Override	Yes	No	No

5. Data Model

The PostgreSQL database is modelled through Prisma and consists of the following core entities:

Entity	Key Fields & Relationships
Store	UUID PK, storeCode (unique), name, ownerPinHash, cashierPinHash, kitchenPinHash, isInitialized
Product	UUID PK, storeId (FK), name, price, stock, minStock, isActive, category
RawMaterial	UUID PK, storeId (FK), name, status (AVAILABLE/LOW/OUT), stock, minStock, overrideUntil
Order	UUID PK, storeId (FK), status (PENDING→COMPLETED), total, createdAt
OrderItem	UUID PK, orderId (FK), productId (FK), quantity, price (snapshot)
Account	UUID PK, storeId (FK), name, type (CASH/BANK/UPI/WALLET), balance
Transaction	UUID PK, storeId (FK), accountId (FK), orderId (FK unique), type (INCOME/EXPENSE), amount, note
OverrideLog	UUID PK, storeId/productId/rawMaterialId (FK), role, reason, createdAt

Enums used: Role (OWNER, STAFF, CASHIER, KITCHEN), OrderStatus (PENDING, PREPARING, READY, COMPLETED, CANCELLED), AccountType (CASH, BANK, UPI, WALLET), TransactionType (INCOME, EXPENSE), RawMaterialStatus (AVAILABLE, LOW, OUT).

6. API Reference

All endpoints are prefixed /v1/. Protected routes require Authorization: Bearer <JWT>. The server returns JSON; error responses include a message field.

Authentication (/v1/auth)

Method	Endpoint	Description	Access
POST	/setup	Initialise store & set owner details	Public
POST	/check	Verify if a store code exists	Public
POST	/unlock	Authenticate user via PIN & Role	Public

POST	/reset-pin	Reset PIN for specific roles	Owner

Products (/v1/products)

Method	Endpoint	Description	Access
GET	/	List all active products	Authenticated
POST	/	Create a new product	Owner
PUT	/:id	Update product details	Owner
DELETE	/:id	Soft delete / archive product	Owner

Inventory (/v1/inventory)

Method	Endpoint	Description	Access
GET	/	List raw materials and statuses	Authenticated
POST	/	Add new raw material	Owner
PATCH	/:id	Update raw material status	Authenticated

Orders (/v1/orders)

Method	Endpoint	Description	Access
GET	/	Fetch orders (filter by status/date)	Authenticated
POST	/	Create a new empty order	Cashier / Owner
POST	/:id/items	Add items to an order	Cashier / Owner
PATCH	/:id/status	Update order status	Kitchen / Owner

Accounts & Transactions (/v1/accounts)

Method	Endpoint	Description	Access
GET	/	List store accounts	Owner
POST	/transaction	Record a payment transaction	Cashier / Owner

Reports (/v1/reports)

Method	Endpoint	Description	Access
GET	/revenue	Revenue data with custom date range	Owner
GET	/orders	Order statistics and trends	Owner
GET	/stock	Low-stock and inventory alerts	Owner

7. Security Implementation

- PIN Hashing: All role PINs (Owner, Cashier, Kitchen) are hashed using Bcrypt before storage. Plain-text PINs are never persisted.
- JWT Authentication: On successful PIN unlock, a signed JWT is issued containing the store ID and role. All protected API routes validate this token via the auth.middleware.js before processing any request.
- Role Enforcement: A secondary requireRole.middleware.js layer checks that the authenticated role matches the permission required by the specific endpoint, preventing privilege escalation.

- Store Code Isolation: Each store's data is scoped to its unique Store Code and UUID. Cross-store data access is architecturally impossible at the query level.
- Override Audit Logging: When an Owner overrides an out-of-stock conflict, the event is recorded in the OverrideLog table with timestamp, role, product, and raw material references.
- Environment Secrets: DATABASE_URL, JWT_SECRET, and API base URLs are injected via environment variables — never committed to the repository.

8. Diagrams & Project References

All architectural diagrams are version-controlled in the repository under backend/diagrams/. The following diagrams are available:

- ER Diagram — Entity-relationship model of the PostgreSQL schema.
- Class Diagram — OOP class structure including PaymentStrategy hierarchy.
- Sequence Diagram — Request/response flow for order creation and payment.
- Use Case Diagram — Actor-action mapping for Owner, Cashier, and Kitchen roles.

Interactive whiteboard (Excalidraw) for system architecture:

- **Excalidraw Board:** https://excalidraw.com/#room=b18d71771add5de4b4f0,1l4lNnrEFPMoBuh1d8_iOQ

Repository and deployment:

- **GitHub Repository:** https://github.com/paramkhodiyar/KitchenOS-SDSE
- **Live Application (Frontend):** https://kitchen-os-sdse.vercel.app
- **API Server (Backend):** https://kitchenossdse.onrender.com

9. Setup & Deployment

Prerequisites

- Node.js v18 or higher
- PostgreSQL database (local or hosted, e.g., Render, Supabase)
- npm v9+

Backend

Clone the repository and navigate to the backend directory. Create a .env file with DATABASE_URL (PostgreSQL connection string) and JWT_SECRET. Run npm install, then npx prisma generate and npx prisma db push to initialise the schema. Start the server with npm run dev (development) or npm start (production).

Frontend

Navigate to the frontend directory. Create a .env.local file with NEXT_PUBLIC_API_URL pointing to the backend server URL. Run npm install and npm run dev to start the Next.js development server. For production, run npm run build followed by npm start, or deploy directly to Vercel.

Demo Seed Data

An optional seed script is available at backend/src/scripts/seed-demo.js. Running node src/scripts/seed-demo.js will populate the database with a demo store, sample products, and raw materials for testing purposes.